

National University of Computer & Emerging Sciences

Karachi Campus



E-commerce Store Management System

Project Report

Object Oriented Programming

Section: BCS-2B

Semester: Spring 2026

Submitted By: 25K-0942 Inaaya Khatri

Submitted To: Miss Seema Hashwani

Abstract

This E-Commerce Store Management System is a GUI-based desktop app programmed in C++ using Object Oriented Programming concepts like Inheritance and Abstraction, with the graphical user interface made in Qt 6 library. There are 2 login options, the admin and the customer. The admin can add, make changes, and remove products and also keep track of orders(confirmed, cancelled or pending) placed by customers and keep track of customers. The customer can log in, look at the product options, add products to a cart, and place an order. The customer and the admin use the same app but have different systems for login and registration. Since file handling is used, all data like products, accounts, and orders is saved and stays there even after the program is closed.

Introduction

Most small stores in Pakistan manage things manually with notebooks, paper receipts, or excel files. There are already many store management platforms available like Shopify, WooCommerce, and OpenCart but these are built for online businesses, need internet and are complex and difficult to operate. Some also cost money and have many extra features that a small store will not use.

I built this project to offer a simple solution for these stores. It is a desktop app that works completely offline and covers only what a small store will need, to manage products, let customers browse and place order, and keep a record of all sales and customer orders. I created a GUI in Qt 6 so the app is user friendly compared to a console based program. I also applied OOP concepts like inheritance and encapsulation throughout the project. Through the use of file handling, data is read and stored in the files.

Objectives

- To create a login and register system where the admin and customer have their own separate access and their own pages
- To allow the admin to add change, and remove products
- To allow customers to view products search by category and manage their cart place their order
- A checkout where customers can select a payment method and pay accordingly
- To let customers see their order history
- To give the admin a way to view all orders and update their statuses
- To use file handling with .csv files to save and restore all of the data
- To apply OOP concepts like classes, inheritance and encapsulation
- To build the interface using Qt 6 so the app is user friendly

System Design

System Overview

When the app opens, the first things shown are the login and register options. Both can be used for customer and admin. A special staff code is required to register as an admin. Depending on the credentials entered, the user either goes into the admin side or the customer side. Both are part of the same app but have completely different menus and functions according to the access rules and requirements for each user.

Algorithm

The flow of the program works like this:

Register -> Login -> perform tasks-> logout

Implementation:

Language: C++

GUI: Qt 6

IDE: Visual Studio Code

Build System: Cmake

Data Storage: CSV files

Operating System: Linux (Fedora)

Key Features

- Login as customer and as admin in the same application
- Products have option to search and category filter
- Shopping cart with item management and a running order total
- Select payment method at Checkout
- View orders according to their categories, pending, cancelled and confirmed
- Admin can add, edit, and delete products, see all the orders and update order status
- File handling so all data is saved and available the next time the app is opened

Code Snippet

```
class CustomerPage : public QWidget
{
    Q_OBJECT
```

```
public:
    explicit CustomerPage(QWidget *parent = nullptr);
    ~CustomerPage();

    void setProfile(const QString &username);
    void loadProducts();
    void loadOrders();
    void updateCartView();
    void loadCart(const QString &username);
    void saveCart(const QString &username);
    void clearCart();
    Cart &getCart();
    void applyProductFilters();

    QString getSelectedProductName() const;
    int getSelectedOrderIndex() const;
    QString getSelectedOrderFilter() const;
    void setOrderItems(const QVector<QPair<QString, int>> &orders);
    void setPopularProducts(const QStringList &products);

    void setTheme(bool isDark);
}
```

Sample Output

ShopBuddy

Welcome to ShopBuddy!

Login

Register

☐ Dark Theme

ShopBuddy

Login

Username:

Password:

☐ Show Password

Login as:

Customer

☐ Dark Theme

Login

Back

ShopBuddy

Admin Dashboard

Welcome, c (Admin)

Product Name:carDescription:Price:Quantity:

Add Product

Remove Product

View Orders

Clear Finished Order

Cancel All Orders

Complete All Orders

Confirm Order

Complete Order

Cancel Order

Filter Orders:

All

Delete User

OrdersUsersProductsAnalytics

Orders:

Order #2: a | Products: tan x3, inaa x1 | Total Qty: 4 | Total: \$187.00 | Status: Pending | City: New York | Delivery: \$10.00 | Date:

Order #3: a | Products: cake x2, tan x1 | Total Qty: 3 | Total: \$112.00 | Status: Pending | City: Quetta | Delivery: \$13.00 | Date:

Change Password

☐ Dark Theme

Logout

Conclusion

The project fulfills all the goals in the proposal. The admin and customer successfully perform their actions on different pages separately and all the data is saved in files so that it is not lost. Since I built it in C++ using Qt 6 for the GUI, i was able to use the OOP concepts we learned throughout the semester into a project that solves a real world problem. It also helped me understand how to structure a big program using classes and how to connect a GUI to the backend logic.

Limitation

- This app can only be used on a single computer and data is only stored on that computer instead of the internet or a network of computers
- Payment details method is not real, the system does not check if the card number is valid, nor does it charge any money.
- Product images are not displayed

Future Enhancements

- Allow a feature to upload images for each product, along with the other details
- Store product reviews
- Add a discount or coupon system.
- Use SQL and databases to store information to make it more efficient compared to text files
- Allow a way to print or export order receipts and sales reports.
- Different kinds of admins that have access to different features like product admin, cart admin, etc.

References

- Qt 6 Documentation <https://doc.qt.io/qt-6/>
- C++ Reference <https://en.cppreference.com/>
- CMake Documentation <https://cmake.org/documentation/>